

```
print("Dhwaj Jain S012 AI&DS DL Expt 7")
```

```
Dhwaj Jain S012 AI&DS DL Expt 7
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, TensorDataset
import numpy as np

# 1. Load the Digits dataset
digits = load_digits()
x = digits.data # Input image data (64 features)
y = digits.target # Target (not used directly in VAE, but good to have)

# Preprocess the data
# Normalize pixel values to [0, 1]
x = x.astype('float32') / 16.0 # Digits are 0-16, normalize to 0-1 by dividing by max value (16)

# Convert to PyTorch tensors
x_tensor = torch.tensor(x, dtype=torch.float32)

# Parameters based on user's request
input_size = 64 # Each image has 64 features (8x8)
encoder_hidden_size = 64 # Hidden layer size for encoder
latent_dim = 2 # Latent dimension
decoder_hidden_size = 4 # Hidden layer size for decoder

# 2. Define the VAE Architecture
class VAE(nn.Module):
    def __init__(self, input_size, encoder_hidden_size, latent_dim, decoder_hidden_size):
        super(VAE, self).__init__()

        # Encoder
        # Linear layer for the encoder's first hidden layer (F(net) - relu)
        self.encoder_fc1 = nn.Linear(input_size, encoder_hidden_size) # wt(hidden size, input)

        # Output layers for mean and log variance
        # Means wts (latent size, hiddents) and means bias wts (latent_size)
        self.fc_mu = nn.Linear(encoder_hidden_size, latent_dim)
        self.fc_logvar = nn.Linear(encoder_hidden_size, latent_dim)

        # Decoder
        # Linear layer from latent space to decoder's hidden layer (he3)
        self.decoder_fc1 = nn.Linear(latent_dim, decoder_hidden_size) # wt(hiddn size, latent_dim)

        # Output layer for reconstructed image (input, hiddn size)
        self.decoder_fc2 = nn.Linear(decoder_hidden_size, input_size)

    def encode(self, x):
        # F(net) - relu
        h1 = F.relu(self.encoder_fc1(x))
        # Mean net = mean wts * F(net) + meand bias
        mu = self.fc_mu(h1)
        # Logvar net
        logvar = self.fc_logvar(h1)
        return mu, logvar

    def reparameterize(self, mu, logvar):
        # Z = mean + (0.5*logvar)*epsilon(random std shape)
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std) # random std shape
        return mu + eps * std

    def decode(self, z):
        # F(net) - relu for decoder's hidden layer
        h3 = F.relu(self.decoder_fc1(z))
        # Reconstructed image (sigmoid)
        return torch.sigmoid(self.decoder_fc2(h3))

    def forward(self, x):
        mu, logvar = self.encode(x)
```

```

        z = self.reparameterize(mu, logvar)
        reconstruction = self.decode(z)
        return reconstruction, mu, logvar

# 3. Define the VAE Loss Function
def vae_loss_function(recon_x, x, mu, logvar):
    # Reconstruction loss (Binary Cross-Entropy for pixel values 0-1)
    # L(x, x_pred)
    BCE = F.binary_cross_entropy(recon_x, x, reduction='sum')

    # KL Divergence loss
    # sum(KL(q(z|x) || p(z)))
    # 0.5 * sum(1 + log_var - mu^2 - exp(log_var))
    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())

    return BCE + KLD

# 4. Training setup
# Hyperparameters
batch_size = 500
learning_rate = 1e-3
epochs = 100

# Create DataLoader
dataset = TensorDataset(x_tensor)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

# Instantiate the VAE model
model = VAE(input_size, encoder_hidden_size, latent_dim, decoder_hidden_size)
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# 5. Training Loop
print("Starting VAE training...")
for epoch in range(epochs):
    model.train()
    train_loss = 0
    for batch_idx, (data,) in enumerate(dataloader):
        optimizer.zero_grad()
        recon_batch, mu, logvar = model(data)
        loss = vae_loss_function(recon_batch, data, mu, logvar)
        loss.backward()
        train_loss += loss.item()
        optimizer.step()

    avg_train_loss = train_loss / len(dataloader.dataset)
    if (epoch + 1) % 10 == 0 or epoch == 0:
        print(f"Epoch: {epoch+1}/{epochs}, Average Loss: {avg_train_loss:.4f}")

print("VAE training complete!")

# Example of generating new data (sampling from latent space)
model.eval()
with torch.no_grad():
    # Generate a random latent vector
    num_samples = 5
    sample = torch.randn(num_samples, latent_dim)
    generated_images = model.decode(sample).cpu().numpy()

print(f"\nGenerated {num_samples} images. Shape of generated images: {generated_images.shape}")

# You can visualize these images to see the VAE's generation capabilities
# For example, reshape the 64-feature vector back into an 8x8 image
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 2))
for i in range(num_samples):
    ax = plt.subplot(1, num_samples, i + 1)
    plt.imshow(generated_images[i].reshape(8, 8), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.suptitle('Generated Images from VAE Latent Space')
plt.show()

```

✓ Original vs Reconstructed Digits

```
# Visualize original vs. reconstructed digits
model.eval()
with torch.no_grad():
    # Take a few samples from the dataset
    num_compare_samples = 10
    # Get the first 'num_compare_samples' original images
    original_images = x_tensor[:num_compare_samples]

    # Pass them through the VAE to get reconstructions
    reconstructed_images, _, _ = model(original_images)

    # Convert to numpy for plotting
    original_images_np = original_images.cpu().numpy()
    reconstructed_images_np = reconstructed_images.cpu().numpy()

plt.figure(figsize=(12, 4))
for i in range(num_compare_samples):
    # Plot original image
    ax = plt.subplot(2, num_compare_samples, i + 1)
    plt.imshow(original_images_np[i].reshape(8, 8), cmap='gray')
    plt.title("Original")
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

    # Plot reconstructed image
    ax = plt.subplot(2, num_compare_samples, i + 1 + num_compare_samples)
    plt.imshow(reconstructed_images_np[i].reshape(8, 8), cmap='gray')
    plt.title("Reconstructed")
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.suptitle('Original vs. Reconstructed Digits')
plt.show()
```

```
import matplotlib.pyplot as plt

# Get latent space representations for the entire dataset
model.eval()
with torch.no_grad():
    mu, logvar = model.encode(x_tensor)
    z = model.reparameterize(mu, logvar).cpu().numpy()

# Create a scatter plot of the latent space
plt.figure(figsize=(10, 8))
plt.scatter(z[:, 0], z[:, 1], c=y, cmap='tab10', s=10, alpha=0.7)
plt.colorbar(label='Digit Class')
plt.xlabel('Latent Dimension 1')
plt.ylabel('Latent Dimension 2')
plt.title('Latent Space Distribution of Digits (Colored by Class)')
plt.grid(True)
plt.show()
```

Start coding or [generate](#) with AI.

```
print(f"Final Average Training Loss: {avg_train_loss:.4f}")
```

Final Average Training Loss: 29.9208

```
model.eval()
with torch.no_grad():
    # Get a single batch from the dataloader
    for batch_idx, (data,) in enumerate(dataloader):
        if batch_idx == 0: # Process only the first batch
            recon_batch, mu, logvar = model(data)

            # Calculate BCE
            BCE_loss = F.binary_cross_entropy(recon_batch, data, reduction='sum')

            # Calculate KLD
            KLD_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
```

```
print(f"\nLoss Components for a single batch:")
print(f"Reconstruction Loss (BCE): {BCE_loss.item():.4f}")
print(f"KL Divergence Loss (KLD): {KLD_loss.item():.4f}")
print(f"Total Loss (BCE + KLD): {(BCE_loss + KLD_loss).item():.4f}")
break
```

```
Loss Components for a single batch:
Reconstruction Loss (BCE): 14178.2930
KL Divergence Loss (KLD): 791.5269
Total Loss (BCE + KLD): 14969.8203
```